Manual de usuario: PromptZal

Daren Uziel Zapeta Molina - 202132253

Introducción

PromptZal es un lenguaje para describir, de forma corta y estructurada, el prompt que se le entrega a una herramienta de inteligencia artificial. Un programador escribe un archivo con extensión .pz que indica quién es el agente, qué datos usa y qué debe hacer la IA. El programa que se describe en este manual lee ese archivo, comprueba que esté bien escrito y que tenga sentido, y produce el prompt final ya redactado, listo para copiarlo y pegarlo en Claude Code o en cualquier otra herramienta de IA.

Al compilar un programa, la aplicación realiza estos pasos:

1. Lee el texto del editor y reconoce cada palabra, símbolo y literal del lenguaje (análisis léxico).
2. Comprueba que las piezas estén en el orden correcto (análisis sintáctico).
3. Comprueba que el programa tenga sentido, por ejemplo que no use una variable que nunca se declaró (análisis semántico).
4. Muestra los errores encontrados, si los hay.
5. Genera el promptzal, que es el prompt final en texto, y lo muestra en pantalla.
6. Permite guardar el promptzal y generar los reportes de errores, de tabla de símbolos y del árbol de sintaxis.

Requisitos

- Java JDK 21 o superior.
- Terminal
- Un navegador web, para ver los reportes HTML.
- Conexión a internet al abrir los reportes. Los reportes cargan su hoja de estilos desde internet; sin conexión se abren igual, pero sin diseño.

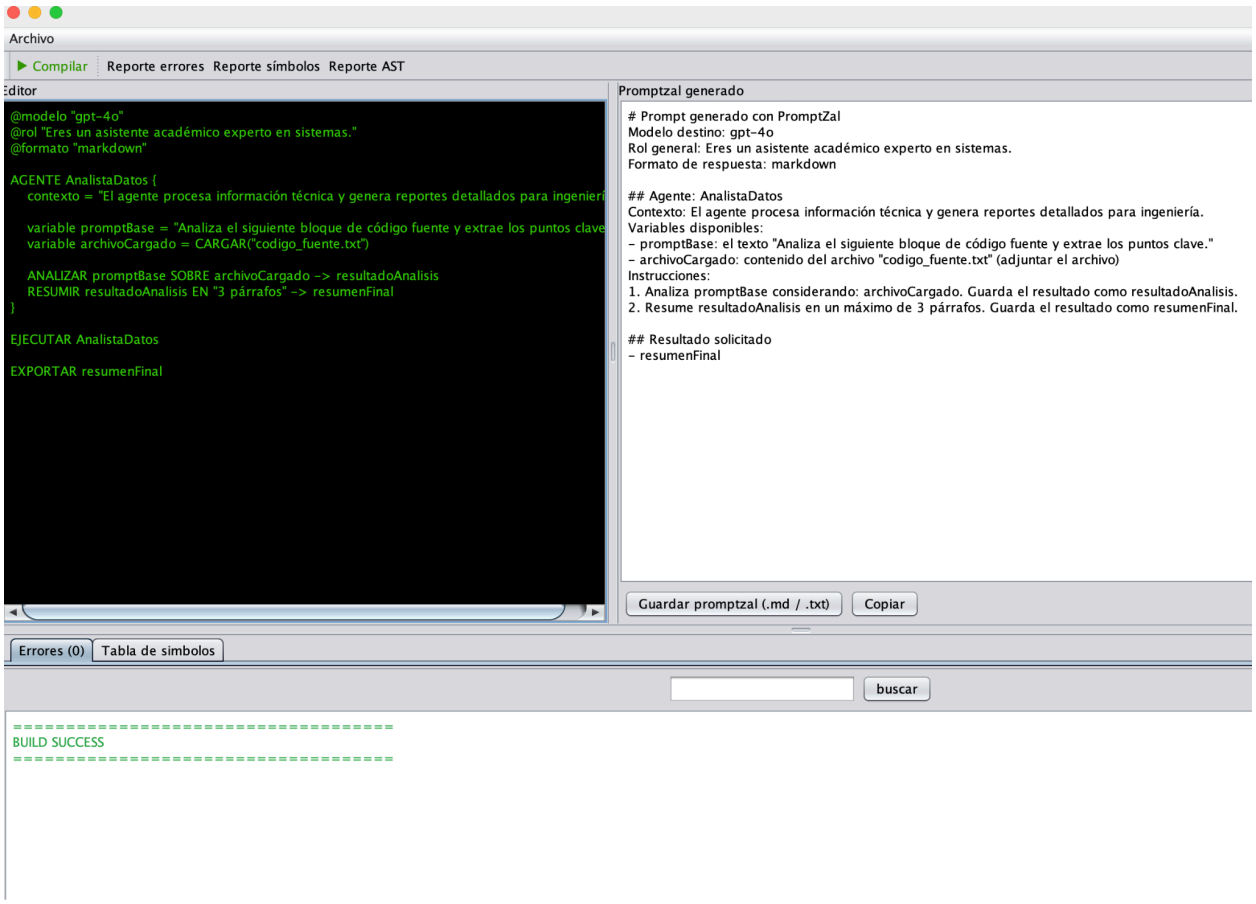
Cómo ejecutar la aplicación

1. Descargar el archivo .jar
2. abrir la terminal
3. Ubica la carpeta donde este el archivo desde la terminal
4. escribir java -jar + nombre del archivo
5. Se abre la ventana principal de la aplicación

Los reportes que genere la aplicación se guardan en una carpeta llamada reportes_html, dentro de la carpeta del proyecto.

Conociendo la interfaz

La ventana principal reúne todo lo necesario para escribir, compilar y revisar un programa. La figura siguiente muestra sus partes; la tabla explica cada una.



Número	Zona	Para qué sirve
1	Menú Archivo	Contiene las opciones Nuevo, Abrir, Guardar y Salir.
2	Barra de herramientas	Tiene el botón Compilar y los botones Reporte errores, Reporte símbolos y Reporte AST.

Número	Zona	Para qué sirve
3	Editor	Es el área donde se escribe o se abre el programa PromptZal.
4	Promptzal generado	Muestra el prompt final que produce la compilación.
5	Botones del promptzal	Guardar promptzal (.md / .txt) guarda el prompt en un archivo. Copiar lo envía al portapapeles.
6	Pestaña Errores	Muestra la lista de errores de la última compilación, con un campo para buscar dentro de ella.
7	Pestaña Tabla de simbolos	Muestra los agentes y las variables del programa, con un campo para buscar.

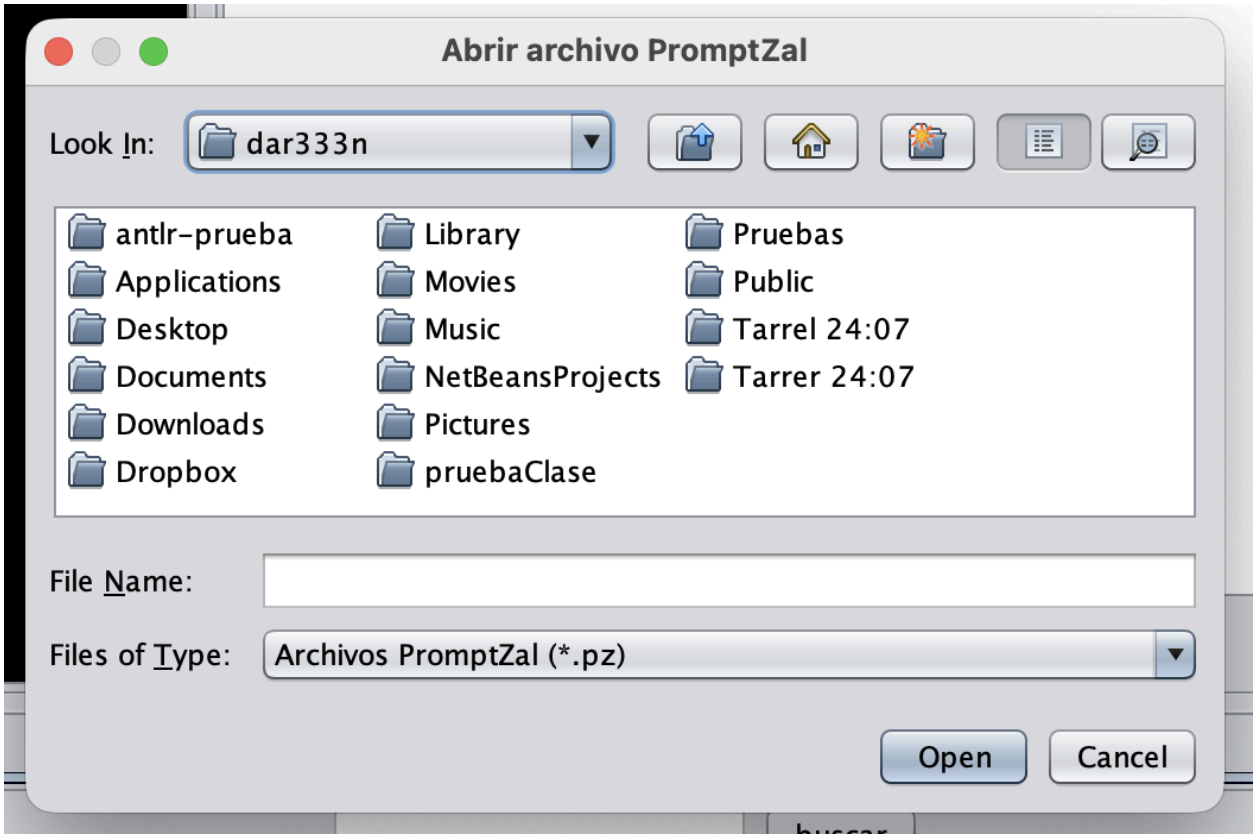
Pasos de uso

Escribir o abrir un programa

Hay dos formas de tener un programa en el editor.

1. Escribirlo directamente en el área Editor.
2. Abrir un archivo existente: en el menú Archivo elija Abrir, seleccione un archivo con extensión .pz y confirme. El cuadro de diálogo solo muestra archivos con esa extensión.





Al abrir un archivo, el contenido anterior del editor y los resultados de la compilación anterior se borran. La opción Nuevo del menú Archivo hace lo mismo sin cargar ningún archivo: deja el editor y todos los paneles vacíos para empezar un programa nuevo.

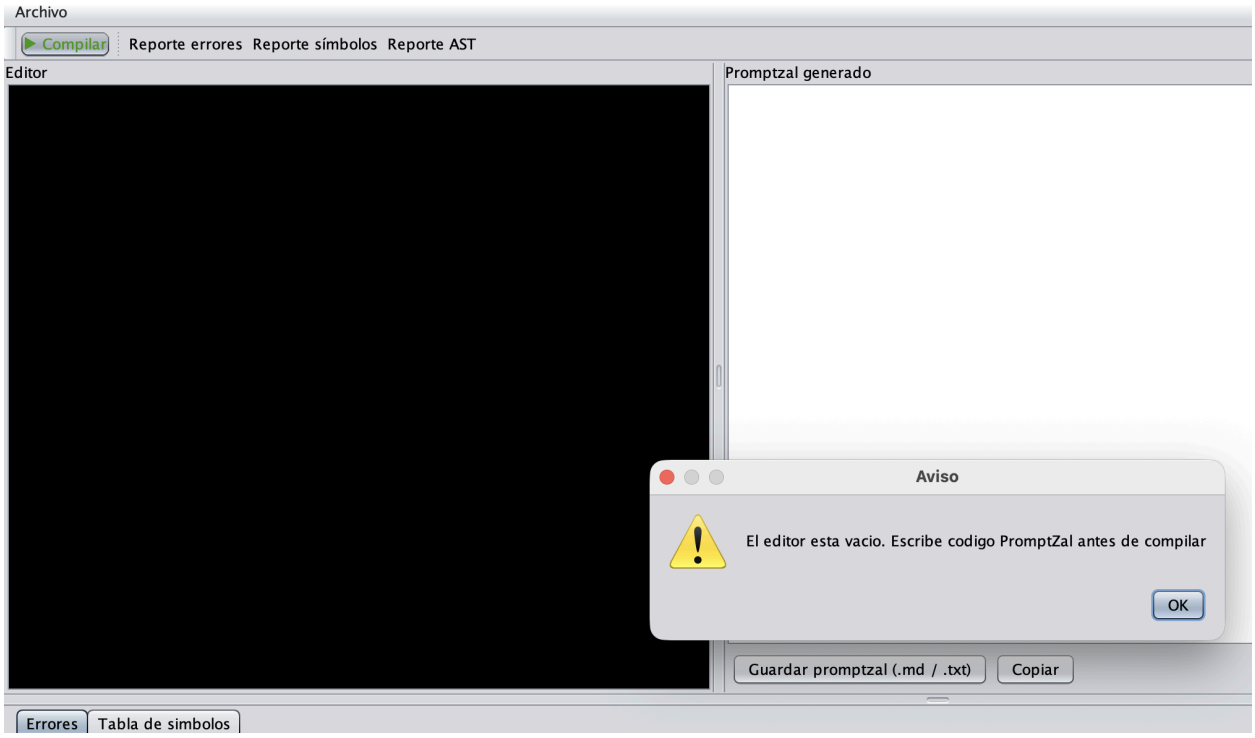
Guardar el programa

1. En el menú Archivo elija Guardar.
2. Si el programa todavía no tiene archivo, la aplicación pide el nombre y la carpeta, y agrega la extensión .pz si no se escribió.
3. Aparece un mensaje con la ruta del archivo guardado.



Compilar

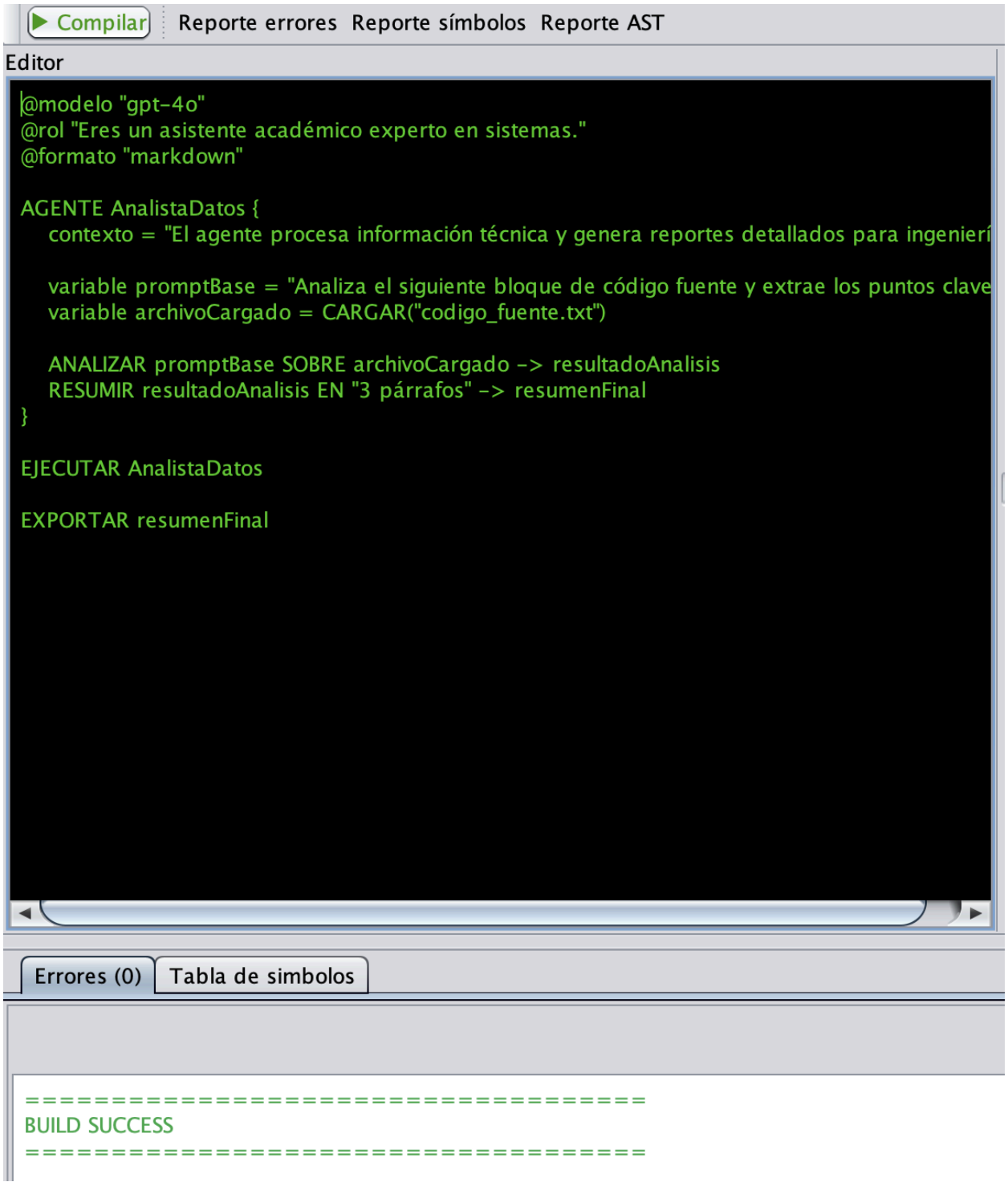
Para analizar el programa, presione el botón Compilar de la barra de herramientas. La aplicación ejecuta todas las fases en orden y muestra los resultados en tres lugares: el panel Promptzal generado, la pestaña Errores y la pestaña Tabla de simbolos.
Si el editor está vacío, aparece el aviso "El editor esta vacio. Escribe codigo PromptZal antes de compilar" y no se compila nada.



Leer el resultado de una compilación correcta

Cuando el programa no tiene errores:

- La pestaña Errores cambia su título a Errores (0) y muestra en verde el mensaje BUILD SUCCESS.
- El panel Promptzal generado muestra el prompt final.
- La pestaña Tabla de simbolos muestra los nombres que declara el programa.

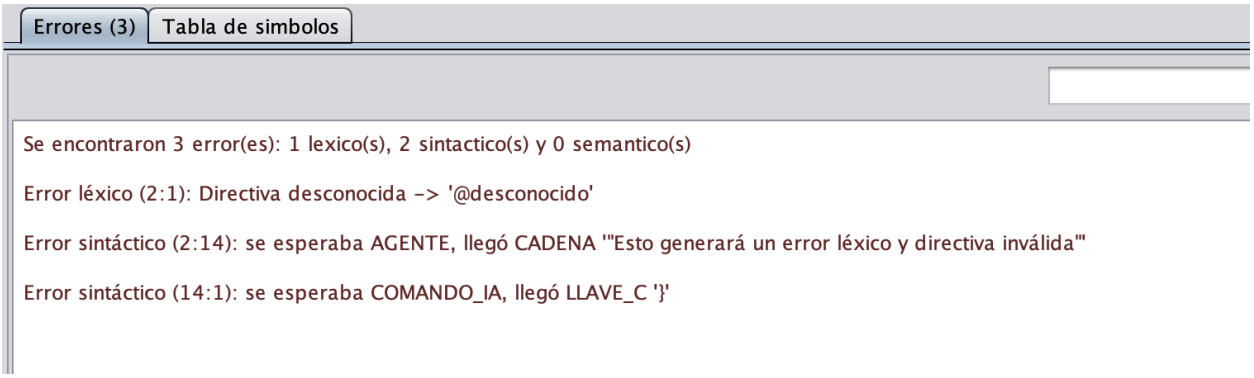


Leer los errores

Cuando el programa tiene errores, la pestaña Errores cambia su título a Errores seguido del total, por ejemplo Errores (3), y el texto aparece en rojo. Primero se indica cuántos errores hay de cada tipo y después se lista cada uno en una línea, con su posición entre paréntesis en el

formato (fila:columna).

Tipo de error	Qué significa	Datos que muestra
Error léxico	Hay una palabra o símbolo que no pertenece al lenguaje.	Posición, descripción y el texto que causó el error.
Error sintáctico	Las piezas del programa no están en el orden que exige el lenguaje.	Posición, lo que se esperaba y lo que se encontró.
Error semántico	El programa está bien escrito, pero no tiene sentido, por ejemplo usa una variable que no existe.	Posición y descripción del problema.



Cada error indica su fila y su columna, de modo que se puede ir directamente a la línea del programa que debe corregirse. La numeración de filas y columnas empieza en 1.
Qué ocurre con el promptzal según el tipo de error:

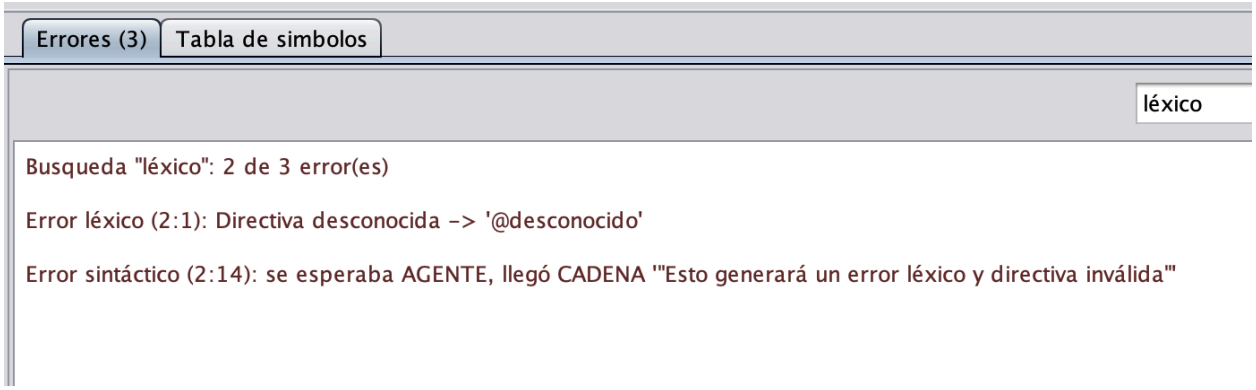
- Con errores léxicos o sintácticos, el panel Promptzal generado muestra el mensaje "No se genero el promptzal (verifique errores de compilacion)" y la tabla de símbolos queda vacía. Hay que corregir esos errores y compilar de nuevo.
- Con errores semánticos, el promptzal sí se genera, pero los errores quedan reportados y conviene corregirlos antes de usar el prompt.

Buscar un error

Si la lista de errores es larga, se puede filtrar:

1. En la pestaña Errores, escriba una palabra en el campo de texto que está junto al botón buscar, por ejemplo semántico, ID o el número de una fila.
2. Presione el botón buscar.
3. El área de texto muestra solo los errores que contienen esa palabra, junto con un resumen como "Busqueda semántico: 3 de 3 error(es)". Si ningún error la contiene, se muestra un mensaje que lo indica.
4. Para ver todos los errores otra vez, deje el campo vacío y presione buscar.

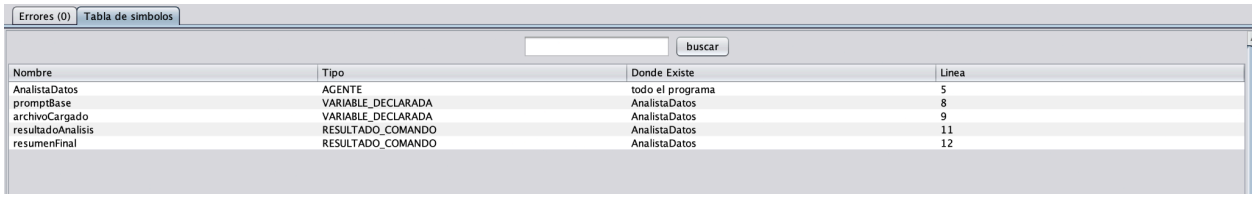
La búsqueda no distingue entre mayúsculas y minúsculas.



Consultar la tabla de símbolos

La pestaña Tabla de simbolos lista todos los nombres que crea el programa. Se llena cuando la compilación no tiene errores léxicos ni sintácticos. Cada fila tiene cuatro datos:

Columna	Contenido
Nombre	El nombre del agente o de la variable.
Tipo	AGENTE para un agente. VARIABLE_DECLARADA para una variable creada con la palabra variable. RESULTADO_COMANDO para una variable que se crea al guardar el resultado de un comando con la flecha.
Donde Existe	El ámbito del nombre. Los agentes existen en todo el programa. Las variables existen solo dentro del agente que las declara y aquí aparece el nombre de ese agente.
Linea	La línea del programa donde se declaró.

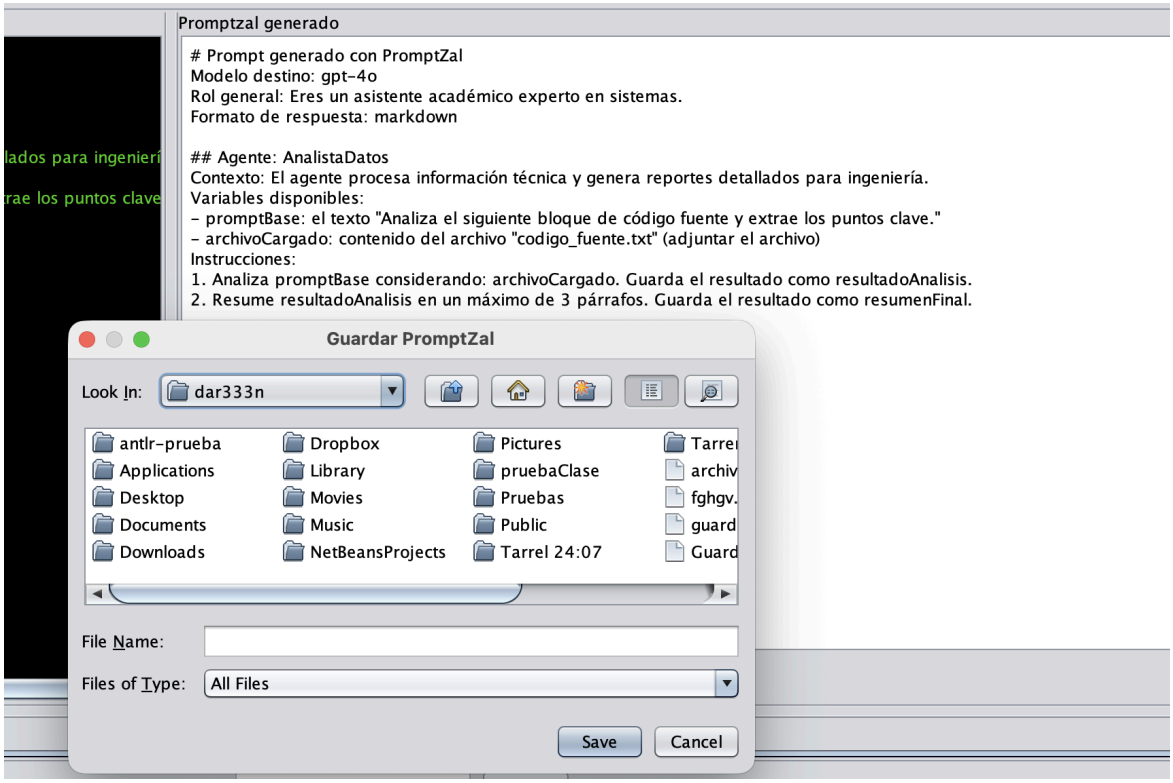


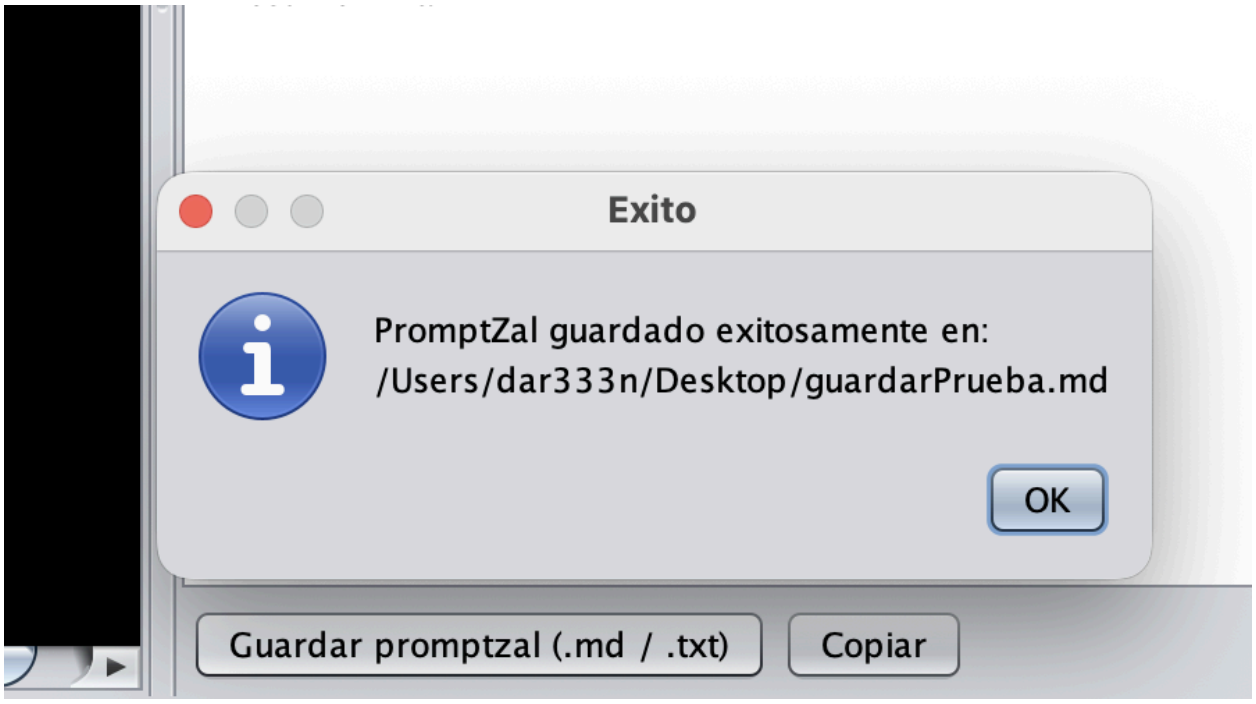
Guardar o copiar el promptzal

Debajo del panel Promptzal generado hay dos botones. Solo funcionan cuando ya se generó un promptzal; de lo contrario aparece el aviso "Todavia no hay un promptzal generado".
Para guardarlo en un archivo:

1. Presione Guardar promptzal (.md / .txt).

2. En el cuadro de diálogo Guardar PromptZal, elija la carpeta y escriba el nombre del archivo. Se puede escoger el formato Markdown (.md) o texto (.txt) en el filtro. Si el nombre no termina en .md ni en .txt, la aplicación agrega .md.
3. Confirme. Aparece un mensaje con la ruta donde quedó guardado.





Para copiarlo al portapapeles, presione Copiar. Aparece el mensaje "Promptzal copiado al portapapeles" y el texto puede pegarse en Claude Code o en cualquier otra herramienta de IA.

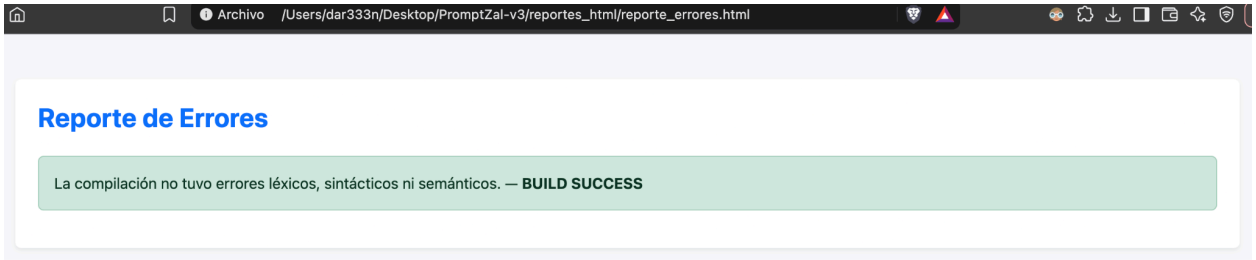


Generar los reportes

Después de compilar, la barra de herramientas permite generar tres reportes. Cada botón crea un archivo HTML en la carpeta reportes_html y lo abre en el navegador. Si el navegador no puede abrirse, la aplicación muestra un mensaje con la ruta del archivo para abrirlo manualmente. Si todavía no se compiló ningún programa, aparece el aviso "Primero compile el programa".

Reporte errores

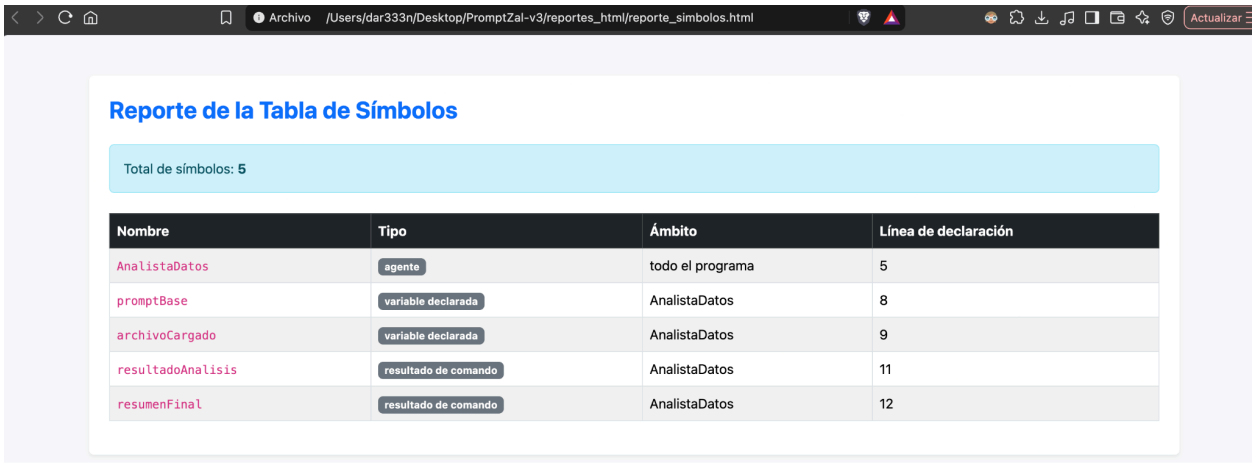
Muestra todos los errores de la última compilación. Indica el total y cuántos son léxicos, sintácticos y semánticos, y los lista en una tabla con las columnas número, tipo, descripción, fila, columna, token encontrado y token esperado. Las dos últimas columnas solo se llenan en los errores sintácticos. Si el programa no tiene errores, el reporte lo indica explícitamente con el mensaje de compilación correcta.





Reporte símbolos

Muestra la tabla de símbolos del programa: total de símbolos y, para cada uno, su nombre, tipo, ámbito y línea de declaración.



Reporte AST

Muestra el árbol de sintaxis abstracta que construyó la aplicación al analizar el programa. El árbol se escribe con sangrías: cada nivel de sangría es un nivel del árbol. Bajo Programa

aparecen las directivas, los agentes (con su contexto, sus variables y sus comandos, cada comando con su argumento, sus conectores y su destino), las instrucciones Ejecutar y la instrucción Exportar. Cada elemento indica la línea donde aparece en el programa.

Reporte del AST

```
Programa
  Directiva @modelo = "gpt-4o"   (línea 1)
  Directiva @rol = "Eres un asistente académico experto en sistemas."   (línea 2)
  Directiva @formato = "markdown"   (línea 3)
  Agente AnalistaDatos   (línea 5)
    Contexto: "El agente procesa información técnica y genera reportes detallados para ingeniería."
    Variable promptBase   (línea 8)
      Valor: "Analiza el siguiente bloque de código fuente y extrae los puntos clave."
    Variable archivoCargado   (línea 9)
      Valor: CARGAR("codigo_fuente.txt")
    Comando ANALIZAR   (línea 11)
      Argumento: promptBase
      Conector SOBRE: archivoCargado
      Destino: resultadoAnalisis
    Comando RESUMIR   (línea 12)
      Argumento: resultadoAnalisis
      Conector EN: "3 párrafos"
      Destino: resumenFinal
  Ejecutar AnalistaDatos   (línea 15)
  Exportar resumenFinal   (línea 17)
```

Guía del lenguaje PromptZal

Estructura de un programa

Un programa PromptZal tiene siempre las mismas partes, en este orden:

1. Directivas, que son opcionales: @modelo, @rol y @formato.
2. Uno o más agentes.
3. Una o más instrucciones EJECUTAR.
4. Una única instrucción EXPORTAR.

Dentro de cada agente el orden también es fijo: primero el contexto, después las variables (puede no haber ninguna) y por último los comandos (al menos uno).

```
@modelo "claude-sonnet-4-6"
@rol "analista de datos"
@formato "markdown"
AGENTE analista {
  contexto = "Eres un analista de datos experto"
  variable ventas = CARGAR("ventas.csv")
  PREGUNTAR "Cuales son las 3 tendencias principales?" SOBRE ventas ->
tendencias
  RESUMIR tendencias EN 100 palabras -> resumen
}
EJECUTAR analista
EXPORTAR resumen
```

Elementos del lenguaje

Elemento	Qué hace	Ejemplo
@modelo	Indica la herramienta o el modelo de IA para el que se redacta el prompt.	@modelo "claude-sonnet-4-6"
@rol	Define el rol general que asumirá la IA en el encabezado del prompt.	@rol "analista de datos"
@formato	Indica el formato en que se desea la respuesta.	@formato "markdown"
AGENTE	Declara un agente: la unidad que agrupa un contexto, variables y comandos. Su cuerpo va entre llaves.	AGENTE analista { ... }
contexto	Define el rol que asumirá la IA dentro del agente.	contexto = "Eres un analista"
variable	Declara una variable y le asigna un valor.	variable ventas = CARGAR("ventas.csv")
CARGAR	Indica un archivo que debe acompañar al prompt. El programa no lee el archivo: solo lo menciona en el promptzal para que se adjunte.	CARGAR("ventas.csv")
Conectores SOBRE, DESDE, EN, COMO	Indican los datos, la fuente, un parámetro o el formato deseado de un comando.	SOBRE ventas
-> (flecha)	Guarda el resultado de un comando en una variable. Esa variable queda declarada.	-> tendencias
+	Une textos y valores en una sola expresión.	"Resumen de " + ventas
EJECUTAR	Indica que un agente se incluye en el prompt final.	EJECUTAR analista
EXPORTAR	Indica qué resultados se piden como salida. Admite varios nombres separados por coma.	EXPORTAR resumen, tendencias

Elemento	Qué hace	Ejemplo
Comentarios	Texto que el programa ignora. Se escriben con // hasta el final de la línea, o entre /* y */.	// esto es un comentario
Cadenas y números	Una cadena va entre comillas dobles. Los números pueden ser enteros o decimales, y un número puede ir seguido de una palabra que lo describe.	"texto", 10, 0.7, 100 palabras

Comandos de IA

Comando	Qué pide a la IA	Conector que usa	Ejemplo
PREGUNTAR	Formula una pregunta sobre unos datos.	SOBRE	PREGUNTAR "Cual es el total?" SOBRE ventas -> total
GENERAR	Genera contenido nuevo: código, texto o ideas.	DESDE	GENERAR codigo DESDE "una calculadora" -> app
RESUMIR	Pide un resumen, opcionalmente con un límite de palabras.	EN	RESUMIR texto EN 100 palabras -> resumen
ANALIZAR	Pide un análisis de un dato bajo cierto criterio.	SOBRE	ANALIZAR ventas SOBRE "estacionalidad" -> analisis
TRADUCIR	Traduce un texto a otro idioma.	EN	TRADUCIR texto EN "ingles" -> traduccion
CLASIFICAR	Clasifica un contenido en un conjunto de categorías.	EN	CLASIFICAR opiniones EN "positiva, negativa" -> clases
EXTRAER	Extrae información específica de un texto.	DESDE	EXTRAER "nombres" DESDE texto -> nombres

El conector COMO puede agregarse a cualquier comando para indicar el formato en que se quiere el resultado, por ejemplo: RESUMIR texto EN 50 palabras COMO "lista" -> resumen. Todo comando termina con la flecha y el nombre de la variable donde se guarda el resultado.

Reglas que debe cumplir un programa

- Las palabras reservadas distinguen entre mayúsculas y minúsculas. AGENTE, EJECUTAR, EXPORTAR, los comandos, CARGAR y los conectores se escriben en mayúsculas. contexto y variable se escriben en minúsculas.
- Los nombres de agentes y variables empiezan con una letra o un guion bajo, y pueden tener letras, dígitos y guiones bajos. No admiten tildes ni la letra ñ. El contenido de una cadena sí puede llevarlas.
- Una variable se declara con la palabra variable o como destino de un comando (después de la flecha). Solo se puede usar después de declararla y únicamente dentro del agente donde se declaró.
- No puede haber dos agentes con el mismo nombre.
- EJECUTAR solo puede nombrar un agente que exista.
- EXPORTAR solo puede nombrar variables que existan en algún agente que se ejecute.
- En GENERAR, si el primer elemento es un nombre, indica el tipo de contenido que se quiere generar (por ejemplo codigo o portada) y no hace falta declararlo como variable.
- La palabra que sigue a un número, como palabras en 100 palabras, solo describe el número y tampoco se declara.